ALGORITHM DESIGN & STRATEGY ENGINE DOCUMENTATION

프리미엄 JavaFX 블랙잭 시뮬레이터

알고리즘 설계 및 전략 엔진 기술 문서

재사용 가능한 규칙 엔진, JavaFX 프레젠테이션 계층, 실시간 전략 분석, 애니메이션 기반 테이블 UX, 세션 영속화, 추천 정확도 분석을 통합한 포트폴리오급 Java 데스크톱 블랙잭 시스템이다.

Java + JavaFX + FXML + CSS

Single-player vs Dealer

Strategy Analysis

Session Persistence

Recommendation Accuracy

작성자: Hyeonjo Kim
문서 유형: Software Engineering Design Document / Technical Whitepaper

문서 범위

- 핵심 객체 모델과 엔진 책임 분리
- 라운드 생명주기와 분기 알고리즘
- 규칙 기반 전략 추천 엔진
- Expected EV, Bust Risk, Dealer Bust Chance 휴리스틱
- JavaFX 상태 동기화와 애니메이션 구조
- 성능 최적화, 세션 영속화, 향후 확장 방향

02. 프로젝트 개요

이 블랙잭 엔진이 해결하는 문제

많은 블랙잭 프로젝트는 단순한 콘솔 플레이에 머무르거나, 규칙 로직이 UI 이벤트 코드에 직접 섞여 있어 확장성과 설명 가능성이 떨어진다. 이 프로젝트는 이러한 한계를 해결하기 위해 게임 엔진, 통계 분석, 전략 추천, JavaFX 프레젠테이션을 명확한 계층으로 분리했다.

그 결과물은 단순히 플레이 가능한 데스크톱 게임이 아니라, 무엇을 추천하는지, 왜 그 결정을 추천하는지, 현재 상태가 얼마나 위험한지 를 실시간으로 설명할 수 있는 알고리즘 시스템이다. 동시에 엔진 자체는 테스트, 시뮬레이션, 다른 UI 플랫폼으로도 재사용 가능하다.

전략 분석과 EV 계산이 중요한 이유

- **Best Move**는 현재 손패와 Dealer Up-card 기준의 최적

GAME ACTIONS

Hit Stand Double Split

단일 split 지원과 multi-hand 정산 흐름을 포함한다.

ENGINE SHAPE

OOP Stateful UI-agnostic

도메인 객체가 렌더링 코드와 분리되어 있다.

ANALYTICS

Session Round Advice

결과 품질과 추천 추종 성과를 동시에 기록한다.

PERSISTE

Save Load Expo

bankroll, theme, aggregate metrics 를 snapshot 복원한다.

핵심 기술 강점. 이 프로젝트는 포트폴리오 앱이 자주 놓치 지점을 공략한다. 설명 가능한 전략 로직, 명시적인 payout 산, split 상태 관리, 라이브 UI 동기화, 세션 영속화, 재사용 능한 애니메이션 헬퍼가 그 핵심이다.

행동을 즉시 제시한다.

- **Expected EV**는 장기 기대 성능을 정규화된 점수로 보여 준다.
- **Bust Risk**는 추가로 한 장을 받을 때 즉시 bust 될 가능성을 수치화한다.
- **Dealer Bust Chance**는 dealer visible card 의 약점을 정량적으로 보여 준다.
- **Recommendation Analytics**는 추천을 따랐을 때와 무시했을 때의 성과 차이를 세션 기준으로 측정한다.

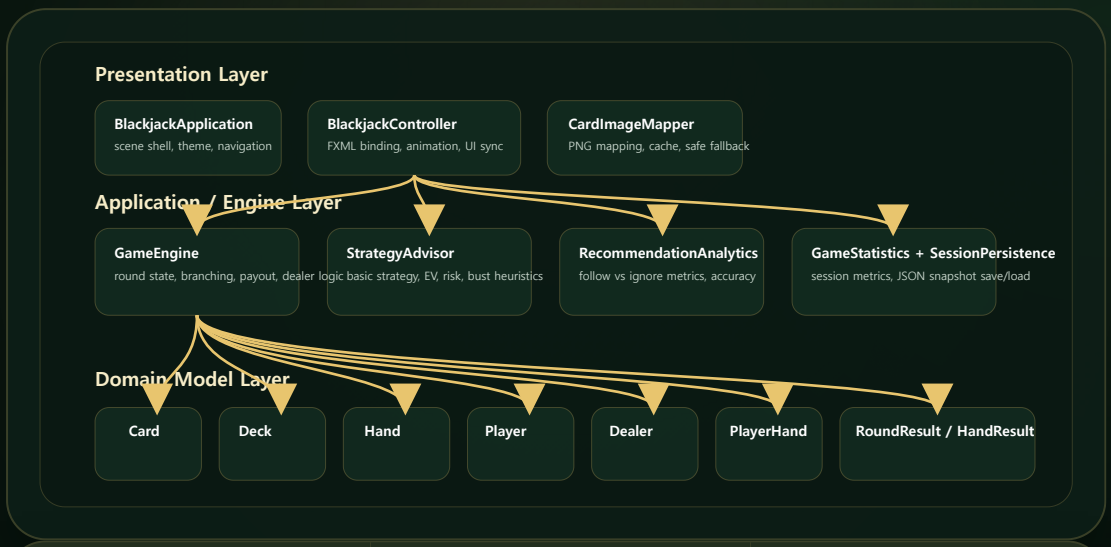
현재 구현 기준의 공학적 전제

- 매 라운드 시작 시 52-card deck 을 새로 생성하고 shuffle 한다.
- dealer draw rule 이 `value < 17` 이므로 soft 17 포함 모든 17에서 st 한다.
- natural blackjack 은 opening deal 직후 즉시 판정한다.
- 현재 natural blackjack payout 은 3:2 가 아니라 `2 × bet` 기반 even-money 정산이다.
- 현재 구현은 라운드당 1회 split 만 허용하며, 이는 multi-hand UI 흐름 단순화한다.

02 / 08

03. 핵심 객체지향 아키텍처

클래스 책임과 계층 구조



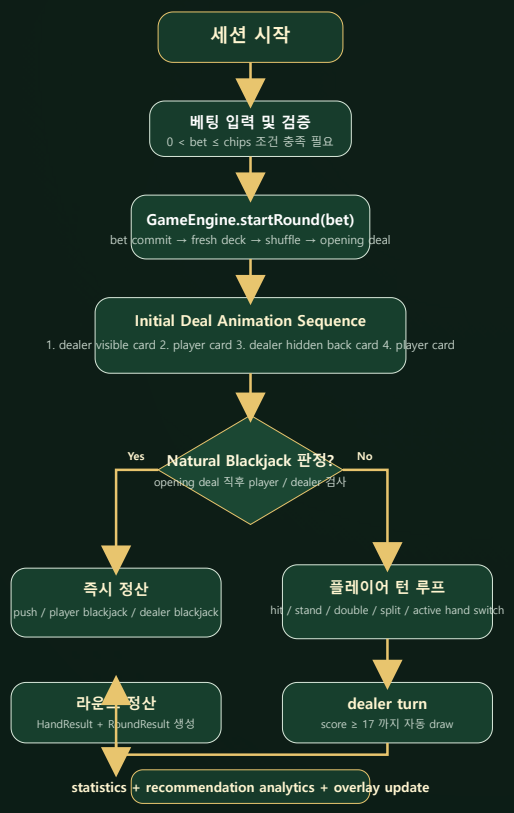
CLASS	주요 책임	설계 포인트
Card	immutable suit/rank 객체와 nominal blackjack value 를 제공한다.	Ace 의 1/11 전환은 Hand 에서 처리하므로 symbolic identity 와 scoring logic 이 분리된다.
Deck	표준 52-card deck 생성, shuffle, draw index 관리.	현재는 round-scoped reset/shuffle 전략을 사용하며 reusable shoe 구조는 아직 도입하지 않았다.
Hand	카드 목록, best value, soft 여부, blackjack, bust, split 가능 여부 계산.	ace downgrade loop 가 scoring correctness 의 핵심이다.
Player	bankroll 보유, bet commit, payout 반영.	chip mutation 을 중앙화하여 UI 중복 정산을 방지한다.
Dealer	dealer hand 와 draw policy 보유.	정책 자체는 매우 단순하며 `value < 17` 조건에 집중한다.
GameEngine	bet, initial deal, action branching, dealer turn, settlement, statistics 전체 오케스트레이션.	게임 규칙의 authoritative boundary 이며 UI 는 이 상태를 읽고 시각화만 수행한다.

CLASS	주요 책임	설계 포인트
`RecommendationAnalytics`	추천 추종 여부와 결과 성과를 hand 단위로 추적한다.	결정 객체를 임시 저장했다가 최종 `HandResult` 와 연결한다.
`StrategyAdvisor`	best move, reason, confidence, EV, bust risk, dealer bust chance 생성.	Monte Carlo 대신 explainable rule-based heuristic 을 사용한다.
`RoundResult`	라운드 outcome, chip delta, total bet, total payout, per-hand detail 을 캡슐화한다.	UI overlay, statistics, analytics 로 전달되는 표준 결과 객체다.
`SessionStatistics`	실제 구현은 `GameStatistics` 와 snapshot DTO 들로 구성된다.	wins, losses, pushes, blackjacks, split/double usage, streak, session P/L 을 관리한다.

03 / 08

04. 게임 라운드 알고리즘 흐름

라운드 생명주기와 분기 구조



라운드 단계별 처리 순서

1. **bankroll / bet 검증.** 플레이어는 opening stake 를 commit 할 수 있어야 한다.
2. **fresh deck 준비.** 엔진은 새 `Deck` 을 생성한 뒤 shuffle 한다.
3. **opening hand 생성.** 하나의 `PlayerHand` 를 만들고 player 2장, dealer 2장을 deal 한다.
4. **natural blackjack 검사.** 일반 턴으로 들어가기 전에 즉시 결과를 정산한다.
5. **플레이어 행동 루프.** active hand 는 hit, stand, double down, split 중 가능한 행동만 선택할 수 있다.
6. **multi-hand 순차 진행.** split 된 손패는 순차적으로 처리되며 active hand index 가 다음 미완료 손패로 이동한다.
7. **dealer turn.** 살아남은 player hand 가 하나라도 있으면 dealer 가 규칙에 따라 draw 한다.
8. **settlement.** 각 손패는 `HandResult` 로, 라운드 전체는 `RoundResult` 로 정리된다.
9. **세션 반영.** statistics, recommendation analytics, overlay, chip

balance, labels 가 모두 결과 객체 기준으로 갱신된다.

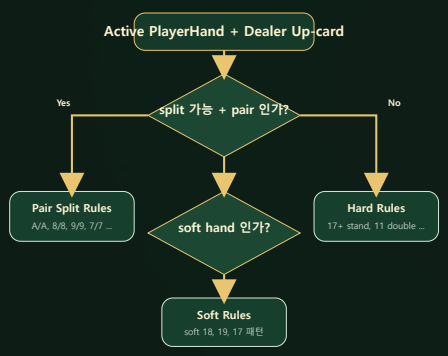
엔진-우선 상태 갱신. 규칙 상태는 항상 엔진이 먼저 확정하고, UI 는 이미 확정된 상태를 기준으로 애니메이션을 수행한다. 이 접근은 시각 효과가 게임 규칙을 오염시키지 않도록 보장한다.

정산 계약(PAYOUT CONTRACT)

```
B = hand 별 committed bet
P = bankroll 로 반환되는 payout
Δchips = P - B

Normal win:        P = 2B,  Δchips = +B
Push:              P = B,   Δchips = 0
Loss / bust:       P = 0,   Δchips = -B
Natural blackjack: 현재 구현도 P = 2B 를 사용
```

05. 전략 추천 알고리즘

의사결정 엔진과 분석 지표 생성 과정



추천 생성 파이프라인

1. 현재 active hand 와 dealer visible up-card 를 읽는다.
2. dealer card 를 전략용 값으로 변환한다. face card 는 10, Ace 는 11 로 본다.
3. split 이 가능하고 pair 면, soft / hard total 로 들어가기 전에 split rule 을 우선 평가한다.
4. split 이 아니면 `soft total` 또는 `hard total` 규칙표로 분기한다.
5. 행동과 이유 문자열을 생성한다.
6. confidence, win tendency, risk level, EV, bust risk, dealer bust chance 를 추가 계산한다.

왜 split 규칙을 먼저 평가하는가. pair split 은 단순한 total 비교가 아니라 게임 트리를 두 개의 독립 베팅 분기로 재구성하는 행위다. 따라서 soft / hard total 보다 먼저 판단되어야 한다.

PAIR SPLIT RULES	
A,A / 8,8	항상 split
9,9	dealer 2-6, 8, 9 에서 split
7,7	dealer 2-7 에서 split

SOFT TOTAL RULES	
Soft 20	Stand
Soft 19	6 상대로 Double, 아니면 Stand
Soft 18	3-6 Double, 2/7/8 Stand,

HARD TOTAL RULES	
17+	Stand
13-16	dealer 2-6 에서 Stand, 아니면 Hit
12	dealer 4-6 에서 Stand, 아니면 Hit

PAIR SPLIT RULES	
6,6	dealer 2-6 에서 split
4,4	dealer 5-6 에서 split
2,2 / 3,3	dealer 2-7 에서 split
5,5 / 10-value pair	split 하지 않음

SOFT TOTAL RULES	
	9/10/A Hit
Soft 17	3-6 Double/Hit, 그 외 Hit
Soft 15-16	4-6 Double/Hit, 그 외 Hit
Soft 13-14	5-6 Double/Hit, 그 외 Hit

HARD TOTAL RULES	
11	non-ace 상대로 Double, 아니면 Hit
10	dealer 2-9 에서 Double, 아니면 Hit
9	dealer 3-6 에서 Double, 아니면 Hit
8 이하	Hit

CONFIDENCE 산정

Aces / Eights split, hard 11 double, hard 11 이하의 명확한 hit 와 같이 canonical basic strategy 패턴은 높은 confidence 를 가진다. 경계 조건은 주로 medium confidence 로 처리된다.

RISK / WIN TENDENCY

double down 과 split 은 stake exposure 가 커지므로 risk 가 상승한다. win tendency 는 hand softness, total strength, dealer weakness 를 조합해 favorable, balanced, dealer-favored 로 분류한다.

05 / 08

06. 기대값(EV) 및 버스트 확률 수식

수학적 로직과 정규화 지표 정의

ACE-AWARE HAND VALUE

```
total = Σ
nominalCardValue
aces = count(Ace)

while total > 21 and
aces > 0:
    total = total - 10
    aces = aces - 1

bestValue = total
softHand = 적어도 하나의
Ace 가 11로 유지되는 상태
```

EXPECTED VALUE HEURISTIC

```
EV =
    softBias
  + dealerWeaknessBias
  + totalStrengthBias
  + actionBonus
  - actionBustPenalty
  + 0.42 × DealerBustChance

Clamp(EV, -0.65, +0.65)
```

이 값은 확률론적으로 완전한 exact EV 가 아니라, 설명 가능성과 실시간 UI 응답성을 우선한 normalized engineering heuristic 이다.

BUST RISK

```
BustRisk(hand) =
    Σ [ w(r) × I(
bust(hand + r) ) ] / Σ
w(r)

where:
    w(2..9) = 4
    w(10/J/Q/K bucket) =
16
    w(A) = 4
```

현재 구현은 남은 deck 전체를 exact enumeration 하지 않고, rank-weight 기반의 단일 draw simulation 으로 즉시 위험도를 계산한다.

WIN RATE, SESSION PROFIT, RECOMMENDATION ACCURACY

```
ResolvedHands = wins + losses + pushes
WinRate = wins / ResolvedHands

SessionProfit = Σ roundChipDelta

FollowRate =
totalRecommendationsFollowed /
totalRecommendationsShown

RecommendationAccuracy =
    (followedWins + followedPushes) /
resolvedFollowedCount
```

DEALER BUST CHANCE

```
DealerBustChance(upCard)
= lookup(upCard)
```

PAYOUT AND BANKROLL INTEGRITY

```
On round start:
    player.commitBet(B)

On split / double:
    player.commitBet(additionalStake)

At settlement:
```

```
2 → 0.35   3 → 0.37   4
→ 0.40   5 → 0.42   6 →
0.42
7 → 0.26   8 → 0.24   9
→ 0.23   10 → 0.21   A →
0.17
```

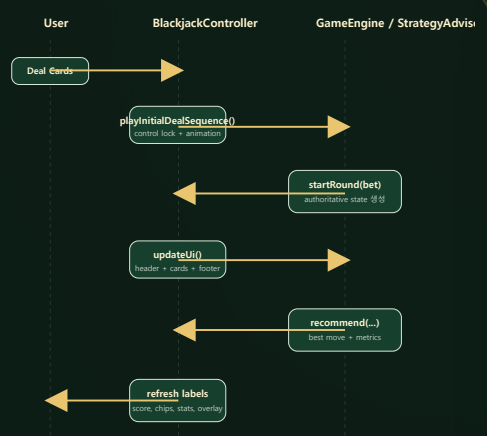
```
player.receivePayout(totalPayoutAmount)

ChipDelta = totalPayoutAmount -
totalCommittedBet
```

bankroll 정산은 `finishRound(...)` 에서 한 번만 수행되므로 UI 중복 반영이나 race-like chip 오염을 방지한다.

지표	의미	공학적 트레이드오프
Expected EV	추천 행동의 장기 기대 성능을 방향성 있게 나타내는 정규화 점수.	실시간 UI 에 충분히 빠르지만 exact combinatorial EV 는 아니다.
Bust Risk	다음 한 장을 받을 때 즉시 bust 될 확률.	remaining deck exact composition 대신 weighted rank model 을 사용한다.
Dealer Bust Chance	dealer up-card 기준 약세를 빠르게 표현하는 lookup probability.	constant-time 이며 설명성이 높다.
Confidence	해당 추천이 얼마나 canonical basic strategy 패턴에 가까운지 나타냄.	통계적 confidence interval 이 아니라 사람이 읽기 쉬운 categorical 신뢰도다.

07. UI 상태 동기화

JavaFX 프레젠테이션을 엔진 상태와 정합시키는 방식



동기화 전략

- **engine-first state.** 규칙 상태는 엔진이 먼저 결정하고 UI 는 그 결과를 시각화한다.
- **단일 갱신 허브.** `updateUi()` 가 header, card area, recommendation panel, footer, control state, outcome effect 를 통합 갱신한다.
- **animation gating.** `animationInProgress`, `initialDealAnimationInProgress` 플래그로 중간 상태 입력을 차단한다.
- **split-hand awareness.** active hand index 를 기준으로 highlight, score, recommendation context 가 함께 이동한다.
- **resolved-state handoff.** 라운드 종료 후 recommendation panel 은 live advice 대신 안전한 "Round complete" 상태로 전환된다.

UI 대상	갱신 시점	SOURCE
card dealing animation	opening deal, hit, dealer draw, dealer reveal	엔진 손피 animatio
chip balance	bet commit, clear bet, settlement, reset,	`Player.` animatio

UI 대상	갱신 시점	SOURCE
	session load	
hand score label	initial deal finish, hit completion, dealer reveal/draw	`Hand.ge`
recommendation panel	initial deal, every hit, split, hand switch, double, dealer reveal, round resolve, session load	`Strateg`
session save/load	utility button click	snapshot serializer

07 / 08

08. 성능 최적화

계산 비용, 렌더링 비용, 확장 비용 관점의 개선 여지

현재 성능 특성

- round 마다 fresh deck + shuffle 을 사용해 상태 추론이 단순하다.
- PNG card image 는 첫 로드 이후 memory cache 를 활용한다.
- dealer bust chance 는 constant-time lookup 이다.
- bust risk 는 full permutation 대신 rank-weight model 로 계산된다.

최적화 기회

- `(hand signature, dealer up-card, action)` 기준 EV / risk memoization.
- per-round deck rebuild 대신 multi-deck shoe 도입.
- re-split 도입 시 split recursion cache 추가.
- 복수 손패 UI 에서 node refresh batching 개선.

JAVAFX 렌더링 최적화

- dedicated animation layer 로 transient node 가 전체 layout 을 흔들지 않도록 분리.
- 고정 card size 유지로 relayout cost 최소화.
- runtime SVG 대신 cached raster PNG 사용으로 decode 비용 제거.
- felt/background 와 dynamic overlay 를 별도 layer 로 유지.

09. 향후 확장 로드맵

전략적 확장 시나리오

로드맵 항목	기술적 가치	확장 방향
Monte Carlo simulation mode	보다 정확한 확률 표면과 고급 EV 계산 제공.	기존 recommendation interface 뒤에 simulation worker 계층 추가.
reinforcement learning strategy	basic strategy 와 학습 정책을 비교 분석할 수 있다.	pluggable policy engine 구조와 offline training data pipeline 도입.
multiplayer support	single player 엔진을 seat-based table state 로 일반화.	`PlayerHand` ownership 과 turn sequencing 을 확장.
online leaderboard	제품성, ranking, 장기 세션 분석 기능 강화.	snapshot 을 backend service 또는 local database 로 저장.

로드맵 항목	기술적 가치	확장 방향
probability heatmap	recommendation confidence 와 EV 를 시각적으로 탐색 가능.	strategy output 을 matrix / overlay 로 렌더링.
mobile version	엔진 재사용성과 cross-platform 설계 완성도 검증.	engine / analytics 계층은 유지하고 mobile UI shell 만 교체.

10. 최종 엔지니어링 요약

면접과 포트폴리오에서 강하게 전달되는 이유

이 프로젝트는 단순한 카드 게임 구현을 넘어, 상태 기계, 규칙 엔진, 추천 로직, 통계 분석, 영속화, 애니메이션 UI 를 한 시스템으로 통합했다는 점에서 완성도 높은 소프트웨어 엔지니어링 사례로 읽힌다.

특히 중요한 점은 카드가 보인다는 사실이 아니라, 시스템이 **의사결정을 설명하고**, **사용자 선택의 결과를 측정하며**, **애니메이션과 규칙 상태를 분리하고**, **의미 있는 세션 상태를 저장/복원** 할 수 있다는 데 있다. 이 특성은 GitHub 포트폴리오, 시스템 설계 인터뷰, UI/엔진 분리 설계 설명 모두에 강한 설득력을 제공한다.

ARCHITECTURE

Reusable Engine

규칙과 JavaFX 렌더링이 명확히 분리되어 있다.

ANALYSIS

Explainable Strategy

모든 추천은 이유와 수치 지표를 함께 제공한다.

PRESENTATION

Premium Desktop UX

카지노 감성의 시각 디자인과 애니메이션, overlay, session utility 를 포함한다.

구현 정합성 메모: 본 문서는 현재 코드 기준을 그대로 반영하며, even-money blackjack payout, 라운드당 단일 split, heuristic EV / risk 모델을 전제로 한다.